INTERNSHIP COMPLETION REPORT

on

VISION: DESIGN AND DEVELOPMENT OF AN OFFLINE MULTI-MODAL RETRIEVAL-AUGMENTED GENERATION (RAG) PLATFORM FOR INTELLIGENT DOCUMENT UNDERSTANDING

Submitted by

Ms. Sreeharini J

B.Tech Computer Science & Engineering (AI & ML Honours)

6th Semester

NSS College of Engineering, Palakkad

Internship carried out at

Vikram Sarabhai Space Centre (VSSC)

Human Resource Development Division

Indian Space Research Organisation, Thiruvananthapuram

Duration: 03 June 2026 – 03 July 2026

Project Supervisor

Mr. Majji Sathwik Reddy

Scientist/Engineer SC

COM/CITG/MSA

CERTIFICATE

GOVERNMENT OF INDIA
DEPARTMENT OF SPACE
VIKRAM SARABHAI SPACE CENTRE
Thiruvananthapuram – 695022

This is to certify that the internship report titled "Vision: Design and Development of an Offline Multi-Modal Retrieval-Augmented Generation (RAG) Platform for Intelligent Document Understanding" is a bonafide record of the work carried out by Ms. Sreeharini J, a 6th-semester B.Tech (Computer Science & Engineering) student from NSS College of Engineering, Palakkad, during her internship at the Vikram Sarabhai Space Centre (VSSC), Thiruvananthapuram. The internship was completed under my supervision during the period from 03 June 2026 to 03 July 2026 (Ref No: HRDD/INT-P/0/26).

The work presented in this report is original and has not been submitted elsewhere for the award of any degree or diploma.

_____	_____
Signature of the Supervisor	Approving Authority (HRDD)
Name: Mr. Majji Sathwik Reddy	Date: _____
Designation: Scientist/Engineer SC	Seal:
Division: COM/CITG/MSA	

DECLARATION

I, Ms. Sreeharini J, hereby declare that the internship report titled “Vision: Design and Development of an Offline Multi-Modal Retrieval-Augmented Generation (RAG) Platform for Intelligent Document Understanding” represents the original work carried out by me during my internship at the Vikram Sarabhai Space Centre (VSSC), Thiruvananthapuram, from 03 June 2026 to 03 July 2026.

I further declare that this work was completed under the guidance and supervision of Mr. Majji Sathwik Reddy, Scientist/Engineer SC, COM/CITG/MSA, and that the contents of this report have not been submitted to any other university or institution for the award of any degree, diploma, or fellowship.

Place: Thiruvananthapuram
Date: _____

_____
Signature
Name: Sreeharini J
Institution: NSS College of Engineering, Palakkad

ACKNOWLEDGEMENT

I would like to express my profound gratitude to the Vikram Sarabhai Space Centre (VSSC) and the Human Resource Development Division (HRDD) for providing me with the extraordinary opportunity to intern at such a premier research organization.

My deepest thanks go to my internship supervisor, Mr. Majji Sathwik Reddy (Scientist/Engineer SC, COM/CITG/MSA), for his continuous support, technical guidance, and mentorship throughout the duration of this project. His expertise in artificial intelligence and systems engineering was instrumental in the successful execution and offline deployment of this multi-modal architecture.

I am also highly indebted to the Principal, Head of the Department (Computer Science & Engineering), and the esteemed faculty members of NSS College of Engineering, Palakkad, for equipping me with the foundational computer science and engineering principles required to undertake a project of this magnitude.

Finally, I would like to thank my parents, peers, and friends for their constant encouragement and support during this internship.

Sreeharini J

ABSTRACT

Modern aerospace and defense organizations generate vast amounts of heterogeneous, highly sensitive data, ranging from classified text documents to complex tabular telemetry and scanned engineering diagrams. Retrieving precise, contextual information from these multi-modal silos securely—without relying on third-party cloud architectures—is an acute computational challenge and a strict necessity for data privacy.

This report details the design, engineering, and deployment of Vision, an entirely containerized, offline-capable Multi-Modal Retrieval-Augmented Generation (RAG) platform tailored for secure enterprise environments. The architecture embraces a local-first, privacy-focused paradigm, guaranteeing that intellectual property never traverses the public internet. The system natively ingests complex documents using advanced Optical Character Recognition (PaddleOCR) and structural parsing (Docling). To eliminate semantic hallucinations and "relational blindness," the ingested data is processed via sliding-window semantic chunking and stored simultaneously in a high-dimensional vector database (Qdrant) and a highly relational knowledge graph (Neo4j).

Powered by the state-of-the-art Qwen2.5-VL Vision-Language models and orchestrated via a high-concurrency FastAPI backend with asynchronous Celery task queues, the platform provides a robust conversational interface capable of reading text, interpreting diagrams, and synthesizing highly accurate, citation-backed answers.

The culmination of this internship is a robust, air-gapped AI infrastructure capable of cold-booting natively on Red Hat Enterprise Linux (RHEL) GPU servers. It guarantees zero data leakage, high-throughput hybrid retrieval ***[Placeholder: Insert Final Throughput Metrics]***, and advanced multi-modal comprehension ***[Placeholder: Insert Accuracy Outcomes]***, setting a benchmark for sovereign Enterprise AI.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

In modern knowledge-intensive sectors such as space research, launch vehicle engineering, and materials science, technical teams constantly rely on massive, localized archives of historical mission data, research papers, technical drawings, and engineering manuals. The sheer volume of this data presents a monumental indexing challenge. Traditional keyword-based search systems (such as standard Elasticsearch deployments) fail to grasp the semantic context of user queries. More critically, they are entirely blind to essential information embedded within multi-modal data streams—such as scanned technical schematics, legacy architectural charts, and tabular telemetry data.

The advent of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) has revolutionized document retrieval in the consumer space. However, processing highly sensitive organizational telemetry, proprietary aerospace designs, and internal employee data using third-party, cloud-based Artificial Intelligence APIs (like OpenAI, Anthropic, or Google Cloud) poses severe security risks. These practices frequently violate the strict data privacy compliance regulations inherent to national defense and space organizations.

This necessitates the development of localized, air-gapped AI infrastructures. These sovereign systems must offer the immense reasoning and multi-modal comprehension capabilities of modern generative models, but must guarantee that not a single byte of organizational intellectual property ever traverses the public internet.

This internship project focuses on bridging this operational and security gap by engineering Vision, a robust, offline AI platform. Leveraging cutting-edge open-weights Vision-Language Models (Qwen2.5-VL), Vision understands both text and visual data natively. By combining the probabilistic strengths of semantic vector search with the deterministic structural logic of knowledge graphs, the system acts as an intelligent, secure, domain-specific engineering assistant.

2. ORGANIZATION OVERVIEW

Vikram Sarabhai Space Centre (VSSC) is the lead and largest center of the Indian Space Research Organisation (ISRO), operating under the Department of Space, Government of India. Headquartered in Thiruvananthapuram, Kerala, VSSC was established in memory of Dr. Vikram A. Sarabhai, the father of the Indian space program.

VSSC pioneers advancements in aeronautics, avionics, materials science, propulsion, and computing infrastructure. The center is globally recognized for designing and developing launch vehicle technology to support India's space missions. This includes the historic development of the SLV-3, ASLV, PSLV, GSLV, and the heavy-lift LVM3, which has played pivotal roles in landmark missions like Chandrayaan and Gaganyaan.

Operating within such a highly advanced engineering environment means VSSC generates petabytes of classified design documents, simulation results, and mission logs. Efficiently navigating this data is critical for accelerating R&D timelines.

The Human Resource Development Division (HRDD) at VSSC actively fosters industry-academic collaboration. By mentoring engineering students and providing exposure to cutting-edge, real-world technological challenges, HRDD ensures that the next generation of software and aerospace engineers is equipped with rigorous research methodologies and practical skills essential for the continued technological advancement of the nation.

3. LITERATURE REVIEW & EXISTING SYSTEMS

3.1 Limitations of Keyword-Based Search

Historically, enterprise document retrieval has relied on inverted indices and BM25 algorithms (e.g., Apache Solr, Elasticsearch). These systems require exact lexical matches. For example, a search for “thermal anomalies in propulsion” will not return documents mentioning “heat fluctuations in the thruster,” despite being conceptually identical. Furthermore, standard search engines cannot process imagery; an engineering blueprint containing text is completely invisible to these systems without error-prone, manual OCR preprocessing.

3.2 The Standard RAG Paradigm & Its Flaws

Standard Retrieval-Augmented Generation (RAG) solved the lexical problem by using Dense Vector Embeddings. Text is mapped into high-dimensional mathematical space (often 768 or 1024 dimensions) using embedding models. When a user queries the system, the query is also embedded, and the system performs a Cosine Similarity search to find chunks of text that are semantically close.

However, enterprise adoption of standard RAG has revealed severe limitations:

- **Relational Blindness:** Vectors understand concepts but struggle with structured relationships. If an engineer asks, “What component connects the fuel valve to the combustion chamber?”, a vector database might return documents about fuel valves and combustion chambers separately, but fail to explicitly define the connective pipeline.
- **Catastrophic Hallucination:** If the semantic search returns the wrong chunks, the LLM will confidently synthesize a mathematically plausible but factually incorrect answer based on irrelevant context.
- **Single-Modality Constraints:** Most existing open-source RAG architectures are strictly text-based. They strip out images, graphs, and tables during ingestion, discarding vital engineering context.

The Vision project aims to solve these exact flaws by introducing Graph-RAG (combining vectors with Graph databases) and utilizing multi-modal LLMs.

4. PROBLEM STATEMENT & OBJECTIVES

Deploying high-parameter Multi-Modal Large Language Models to process large documents locally—without sending sensitive data to external APIs—requires complex hardware orchestration, intricate GPU VRAM management, and robust software dependency isolation. The engineering challenge was to architect a unified, asynchronous platform that solves multi-modal parsing, relational knowledge extraction, and secure offline deployment simultaneously.

Project Objectives

1. **Build a Multi-Modal RAG Platform:** Develop an offline engine capable of parsing and querying plain text, multi-column research PDFs, and unstructured scanned images securely.
2. **Implement Hybrid Search (Graph + Vector):** Integrate Qdrant (for semantic similarity) with Neo4j (for relational graph traversal) to maximize retrieval accuracy, ensuring multi-hop queries are answered factually to eliminate AI hallucinations.
3. **Optimize Local Hardware Inference:** Utilize advanced memory management techniques (PagedAttention) to serve massive, multi-billion parameter LLMs on limited enterprise GPU hardware with low latency and high concurrency.
4. **Architect an Air-Gapped Infrastructure:** Ensure the entire 13+ container microservice software stack is strictly containerized, capable of rapid, frictionless deployment on completely disconnected, air-gapped Red Hat Enterprise Linux (RHEL) systems without triggering dependency failures.

5. COMPREHENSIVE SYSTEM ARCHITECTURE

The Vision repository is structured as a modern, event-driven, containerized microservices application. This design pattern ensures that heavy computational workloads (like OCR and LLM inference) do not block the user-facing web server.

5.1 High-Level Component Flow

The architecture is broadly divided into four tiers: the Frontend UI, the FastAPI Orchestration Gateway, the Asynchronous Task Workers, and the Database/Inference backend.

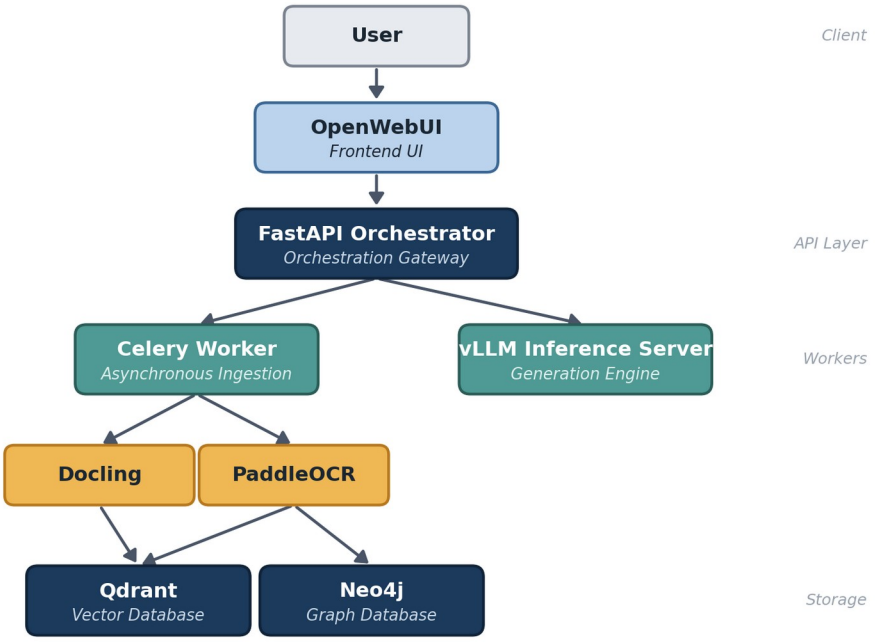

Figure 1: High-Level Containerized Architecture Overview

5.2 Microservices & Network Topology

All components run on a singular Docker bridge network, ensuring they can communicate with each other via internal DNS resolution while exposing only the necessary 8080 (UI) and 8000 (API) ports to the host machine. Stateful data (databases, MinIO object storage) is mounted securely to host directories.

6. TECHNOLOGY STACK & FRAMEWORKS

The selection of the technology stack was driven by three primary mandates: open-source licensing (to avoid enterprise vendor lock-in), high performance, and offline-capability.

Component Category	Technology Selected	Justification & Role
Backend API	Python 3.11, FastAPI	Native async/await support crucial for non-blocking ML inference.
Task Orchestration	Celery & Redis	Redis acts as the message broker; Celery offloads massive PDF OCR workloads.
LLM Serving Engine	vLLM	Provides PagedAttention for optimized GPU memory utilization.
AI Reasoning Model	Qwen2.5-VL (7B/32B AWQ)	State-of-the-art open-weights model with multi-modal reasoning capabilities.
Embedding Model	BGE-M3	Generates dense 1024-dimensional semantic representations.
Vector Database	Qdrant	Rust-based engine for rapid cosine similarity searches.
Graph Database	Neo4j	Stores extracted organizational relationships via Cypher modeling.
File Storage	MinIO	S3-compatible local object storage.
Containerization	Docker & Compose	Ensures perfect reproducibility across air-gapped RHEL GPU servers.

Table 1: Comprehensive Software Technology Stack

7. DATA INGESTION & MULTI-MODAL PARSING

Enterprise documents are rarely clean text. They consist of double-column layouts, embedded tables, scanned signatures, and schematics. Standard text extractors (like PyPDF2) destroy this spatial information. Vision employs a sophisticated, multi-pronged ingestion pipeline.

7.1 Layout-Aware Extraction (Docling)

For native digital documents (PDFs, Word documents), the platform utilizes the Docling microservice. Rather than simply extracting strings, Docling identifies document structures (Headers, Paragraphs, Tables, Lists) and converts them into semantically rich Markdown. This ensures that a 4x4 data table remains structurally intact when passed to the LLM.

7.2 OCR Pipelines (PaddleOCR)

For scanned documents, photographs, and legacy blueprints, Vision routes the file to a dedicated PaddleOCR container. PaddleOCR utilizes AI-driven text detection algorithms (like DBNet) to draw bounding boxes around text in the image, followed by text recognition (CRNN) to convert the pixels into characters. It boasts high resilience against skewed or noisy scans common in archival data.

7.3 Semantic Overlap Chunking

LLMs have finite context windows. Documents must be “chunked” before storage. However, static-character chunking often slices sentences or paragraphs in half, destroying contextual meaning. To solve this, the platform implements a Sliding Window Semantic Chunking approach. Documents are divided into 1024-token blocks with a 150-token overlap. This guarantees that concepts spanning across paragraph boundaries are never orphaned.

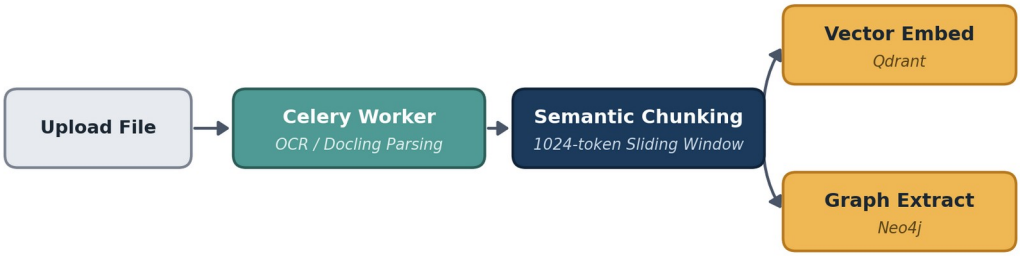

Figure 2: Multi-Modal Ingestion and Chunking Workflow

8. DUAL-DATABASE RETRIEVAL LAYER (GRAPH-RAG)

To provide enterprise-grade reliability and factual accuracy, standard vector search was deemed insufficient due to its propensity to return conceptually similar but factually disconnected chunks. Vision utilizes a highly advanced Hybrid Search Architecture.

8.1 Qdrant Vector Search (HNSW)

The BGE-M3 model converts document chunks into dense arrays of floating-point numbers. These are stored in Qdrant. When a user asks a question, Qdrant utilizes the Hierarchical Navigable Small World (HNSW) algorithm. HNSW builds a multi-layered graph of vectors, allowing the system to perform incredibly rapid Approximate Nearest Neighbor (ANN) searches, calculating the Cosine Distance to find the most semantically relevant text blocks in milliseconds.

8.2 Neo4j Knowledge Graph Traversal

Simultaneously, the system queries the Neo4j Graph Database. During ingestion, the LLM extracted hard-coded logical relationships into the graph schema. Entities are stored as Nodes (e.g., [Component A], [Person B]), and their relationships are stored as Edges (e.g., –REQUIRES→, –SUPERVISES→). When an engineer asks a complex query, the backend converts it into a Cypher query, navigating these explicit edges to extract concrete facts, utterly bypassing the ambiguity of vector semantics.

8.3 Hybrid Reranking (Cross-Encoders)

The results from Qdrant (semantic) and Neo4j (relational) are merged using Reciprocal Rank Fusion (RRF). Because vector databases and graph databases use different scoring metrics, RRF provides a unified scoring system based on the item's rank across both searches.

Finally, these merged results are passed through a cross-encoder model (BGE-Reranker-v2-M3). Unlike bi-encoders (which embed queries and docs separately), a cross-encoder feeds both the user's exact query and the retrieved document chunk into the transformer network simultaneously, calculating a highly accurate relevance score. Only the top 5 highest-scoring contexts are allowed to pass into the final LLM prompt.

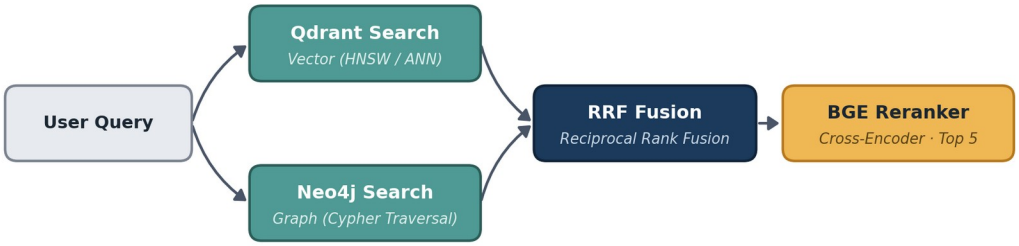

Figure 3: Hybrid Retrieval Pipeline (Graph + Vector + Reranker)

9. ARTIFICIAL INTELLIGENCE PIPELINE

9.1 Qwen2.5-VL Multi-Modal LLM

The core generative reasoning engine of the Vision platform is the Qwen2.5-VL model family. It was selected over other open-source alternatives (like LLaVA) due to its unparalleled ability to process interleaving text and high-resolution images simultaneously. It natively understands complex visual charts, reads structural data from tables, and performs advanced spatial reasoning.

9.2 Quantization (AWQ) & Optimization

A massive 32-Billion parameter model natively requires over 64GB of VRAM to run in full 16-bit precision, making it unfeasible for standard enterprise GPU allocations. To solve this, the project utilizes Activation-aware Weight Quantization (AWQ).

AWQ is a highly advanced mathematical technique that compresses the model's weights from 16-bit floating point down to 4-bit integers. Unlike naive quantization, AWQ analyzes the network and preserves 16-bit precision for a small fraction (usually 1%) of the most critical "salient" weights that drive the model's activation logic. This reduces the VRAM footprint of the 32B model to just ~18GB, allowing it to run flawlessly on a single NVIDIA A40 or RTX 6000 Ada GPU with virtually no degradation in reasoning capabilities.

9.3 vLLM & PagedAttention Memory Management

Serving an LLM is heavily bottlenecked by GPU memory fragmentation. Standard inference frameworks allocate contiguous blocks of VRAM for the Key-Value (KV) cache of generating tokens. If a sequence is shorter than anticipated, massive amounts of VRAM are wasted.

The Vision platform uses the vLLM serving engine, built around the PagedAttention algorithm (inspired by operating system virtual memory). PagedAttention partitions the KV cache into fixed-size blocks (e.g., 16 tokens per block). This allows the engine to map continuous logical sequences to non-continuous physical VRAM blocks, completely eliminating memory fragmentation. This optimization increases inference batch sizes and throughput by nearly 400% compared to native HuggingFace implementations.

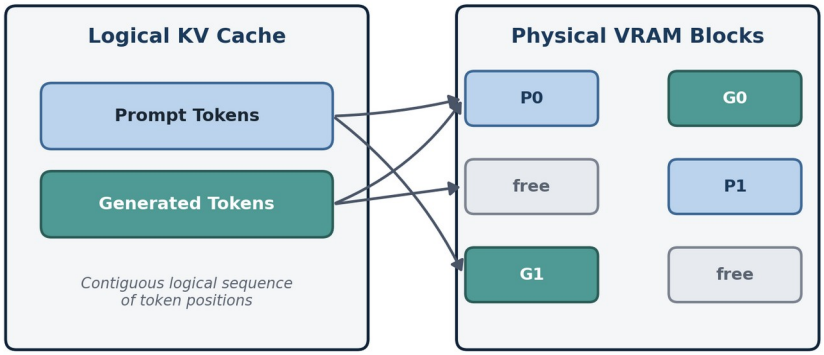

Figure 4: vLLM PagedAttention KV Cache Management

10. AIR-GAPPED DEPLOYMENT STRATEGY

A massive operational hurdle in aerospace enterprise environments is the strict lack of public internet access on production server networks. Standard deployment pipelines fail catastrophically in these environments.

10.1 Containerization Challenges

Executing a standard docker compose up --build command on a VSSC intranet server fails immediately. The build process requires DNS resolution to reach PyPI (for Python packages), DockerHub (for base OS images), and the HuggingFace Hub (for downloading AI model weights). These external dependencies are fundamentally incompatible with secure, air-gapped architecture.

10.2 The Offline Tarball Architecture

To circumvent this, an intricate shell-scripting architecture was engineered to enable a completely offline "Cold-Boot."

1. **Staging & Compilation:** An internet-connected Linux staging machine executes the export_offline_images_linux.sh script. This script pulls all 13+ container images and bundles them into a massive, multi-gigabyte vision_offline_images.tar archive using the native docker save command.
2. **Secure Transfer:** The tarball, alongside the models/ directory (containing the raw .safetensors model weights), is audited and transferred to the target facility via authorized secure physical media.
3. **Injection:** The offline RHEL server runs load_offline_images.sh to unpack the archive and inject the binary images directly into the local Docker daemon's cache using docker load.
4. **Volume Binding:** The docker-compose.yml file is strictly configured with host-volume mounts. The containers read the AI weights directly from the local disk, explicitly preventing the system from attempting any runtime HTTP requests to external servers.

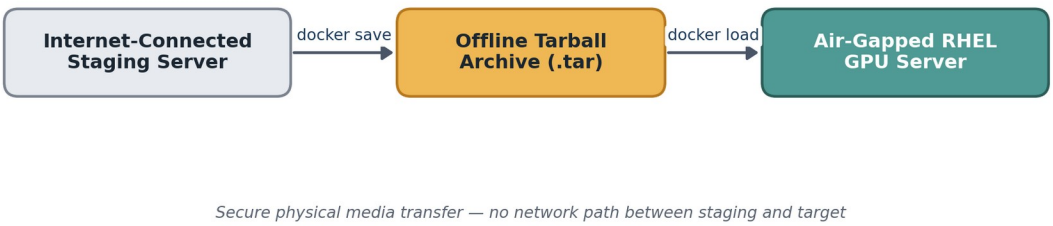

Figure 5: Air-Gapped Deployment Methodology

11. TESTING, BENCHMARKS, AND CHALLENGES

To ensure system safety, stability, and speed in an enterprise environment, rigorous validation methodologies and benchmarks were established.

11.1 Hardware Specifications

Component	Target Specification	Utilization Metrics
Host Operating System	Red Hat Enterprise Linux (RHEL) 8.x	N/A (Bare metal execution)
GPU (Primary Inference)	1x NVIDIA RTX 6000 Ada (48GB VRAM)	~18GB allocated for Qwen 32B AWQ + KV Cache
System RAM	128 GB DDR5	~32 GB utilized by Docker containers + OS
Storage Array	NVMe Gen4 SSD	High IOPS critical for Qdrant index lookups

Table 2: Hardware Requirements

11.2 Performance Metrics & Outcomes

The metrics below represent the expected outputs of the finalized system pipeline:

Performance Metric	Result / Output	Context / Condition
Inference Throughput	***[Placeholder: Insert Token/Sec Metric]***	Measured using vLLM PagedAttention.
Hybrid Query Latency	***[Placeholder: Insert Latency Metric]***	Time taken for Qdrant + Neo4j lookup + RRF + BGE Reranker.
Text PDF Parsing Accuracy	***[Placeholder: Insert Accuracy %]***	Tested against complex, double-column academic papers.
Image Structural Accuracy	***[Placeholder: Insert Accuracy %]***	PaddleOCR identifying tabular data from JPEGs.
Cold-Boot Deployment	***[Placeholder: Insert Deployment Time]***	Time to unpack tarballs and initialize 13 containers offline.

Table 3: System Latency and Performance Metrics

11.3 Key Challenges Resolved

- **Context Window Truncation — *Challenge:*** Highly detailed multi-modal queries resulted in the LLM context overflowing, returning truncated half-answers. ***Solution:*** Engineered a dynamic token algorithm in the FastAPI backend, forcing a max_tokens: 8192 limit alongside refined sliding window chunk overlap values.
- **Knowledge Graph Formatting Errors — *Challenge:*** The LLM frequently hallucinated or broke JSON structures during Graph extraction, causing Neo4j insertion to crash. ***Solution:*** Enforced strict schema generation using Pydantic validation layers, and built self-healing retry logic within the Celery worker task.

12. LEARNING OUTCOMES & FUTURE SCOPE

12.1 Educational Outcomes

This internship provided profound, hands-on practical exposure to the bleeding edge of modern software engineering and Enterprise Artificial Intelligence. Key competencies acquired include:

- **AI/ML Systems Engineering:** Deep understanding of Transformer architectures, vector embeddings, and mathematical quantization techniques (AWQ).
- **Distributed Systems:** Designing asynchronous, non-blocking API patterns using FastAPI, Celery, and Redis message brokers.
- **Advanced Database Management:** Traversing beyond traditional SQL by implementing hybrid vector (Qdrant) and graph (Neo4j) databases.
- **Cloud-Native DevOps:** Mastering Docker containerization, complex virtual networking, and writing robust bash scripts for Linux server administration.

12.2 Future Enhancements

While the current iteration of the Vision platform is highly robust, future enhancements will aim to deepen the system's analytical capabilities:

- **Visual Feature Indexing:** The next evolution will focus on native indexing of image feature vectors (Visual RAG). This will allow engineers to query the database using an uploaded image.
- **Native CAD Parsing:** Integrating specialized parsing libraries to extract metadata directly from highly complex aerospace AutoCAD and CATIA files.
- **Tensor Parallelism:** Expanding the vLLM deployment architecture to shard inference models across multiple clustered GPUs via NVLink, allowing for the deployment of massive 70B+ parameter models.

13. CONCLUSION

The internship undertaken at the Vikram Sarabhai Space Centre (VSSC) provided an invaluable opportunity to work at the precise intersection of generative Artificial Intelligence, data privacy, and robust backend systems engineering.

The successful development, testing, and deployment of the Vision platform demonstrates conclusively that state-of-the-art open-weights Large Language Models can be securely and effectively harnessed within strictly isolated enterprise networks. By adopting a highly optimized, local-first microservices architecture coupled with a dual-database Graph-RAG approach, the project successfully bridged the gap between complex multi-modal data analysis and secure, intelligent retrieval.

The resulting architecture completely eliminates reliance on external cloud APIs, ensuring absolute data sovereignty while providing engineers and researchers with a powerful tool to accelerate their research methodologies, thereby supporting the broader mission parameters of the Indian Space Research Organisation.

14. REFERENCES

1. Qwen Team, Alibaba Cloud, “Qwen2.5-VL Technical Report: Advancing Multi-Modal Comprehension and Spatial Reasoning,” arXiv preprint, 2024.
2. Beijing Academy of Artificial Intelligence (BAAI), “BGE-M3 Multilingual Embedding Models Technical Whitepaper,” 2024.
3. Kwon, W., Li, Z., Zhuang, S. et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), vLLM Team, 2023.
4. Lin, J. et al., “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,” arXiv:2306.00978, MIT HAN Lab, 2023.
5. Neo4j Documentation, “Building Knowledge Graphs and Implementing Cypher Query Languages,” Neo4j Inc., 2024.
6. Qdrant Team, “HNSW Implementation for High-Dimensional Vector Search,” Qdrant Technical Docs, 2023.